

JSON & Python — Complete Beginner Summary

Everything covered in this session, in one place.

1 — What is JSON?

JSON (JavaScript Object Notation) is plain text that stores data in a structured way. Both humans and computers can read it. It is used everywhere — APIs, config files, and databases.

2 — JSON Syntax Rules

1 Keys must always be in double quotes

`"name": "Aisha"`

- Single quotes or no quotes = invalid JSON

2 Only ONE root value is allowed per file

`{"a": 1}` valid `23` true `[1,2]` invalid

- Multiple values need a container: an array or object

3 Wrap multiple values in a container

`[23, true, ["alien", "b"]]`

- Or use an object: `{"num": 23, "flag": true, "items": [...]}`

3 — The 6 JSON Types and Their Python Equivalents

string <code>"hello" -> str</code>	number <code>42 / 3.9 -> int/float</code>	boolean <code>true -> True</code> Python capitalises T
boolean <code>false -> False</code> Python capitalises F	null <code>null -> None</code> Python capitalises N	array <code>[1,2,3] -> list</code> Index starts at 0
object <code>{"k":"v"} -> dict</code>		

4 — The 4 Python JSON Functions

<code>json.loads(text)</code> JSON string -> dict / list loads = "load string"	<code>json.dumps(obj)</code> dict / list -> JSON string indent=2 makes it readable
<code>json.load(file)</code> .json file -> dict / list No 's' — reads from a file object	<code>json.dump(obj, file)</code> dict / list -> .json file indent=2 makes the file readable

5 — Quoting JSON Strings Inside Python

O Single quotes outside — best practice

K `'{"name": "Aisha"}'`
Single quotes outside, JSON double quotes inside — no conflict

O Triple quotes — great for multi-line JSON

K `"""{"name": "Aisha"}"""`
Use triple quotes when JSON spans multiple lines

NOT OK: `"{"name": "Aisha}"` — double quotes clash with JSON, causes `SyntaxError`

6 — Accessing Data — Must Parse First

You CANNOT access JSON directly from a raw string or file. You must call `json.loads()` or `json.load()` first to convert it into a Python dict or list. Only then can you use `["key"]` or `[index]`.

```
# Wrong – json_text is just a string, not a dict
json_text = '{"name": "Aisha", "age": 28}'
print(json_text["name"]) # TypeError!

# Right – parse it first
data = json.loads(json_text)
print(data["name"]) # Aisha

# Nested access – chain brackets
data["user"]["address"]["city"]

# Safe access – avoids KeyError crash
data.get("country", "Unknown") # returns 'Unknown' if key missing
```

7 — JSON File Formats

`.json` (standard)

One root value per file. Use `json.load(file)`

`.jsonl` (JSON Lines)

One JSON value per line. Read line by line with `json.loads(line)`
Not standard JSON — special format

8 — Common Mistakes to Avoid

! Never name your file `json.py`

It shadows Python's built-in `json` module and causes: `AttributeError: module 'json' has no attribute 'loads'`

! `'undefined'` is not a JSON type

Use `null` instead. JavaScript has `undefined` — JSON does not.

!

Python True/False/None vs JSON true/false/null

Python capitalises, JSON does not. `json.loads()` converts automatically between them.

!

Never name your files after standard library modules

Avoid: `json.py`, `math.py`, `os.py`, `random.py`, `datetime.py`, `string.py` — all cause the same import conflict.

Generated from your JSON & Python beginner session with Claude